

낭만포차

QR 기반 술집 모바일 오더 시스템

프로젝트 문서화 · 사업계획서 · 개발 진행도

문서 버전	v1.0
작성일	2026년 5월 22일
프로젝트명	nangman (낭만포차)
레포지토리	github.com/.../nangman [Private]
배포 환경	Vercel (프론트) + Railway (API / PostgreSQL)

목차

- | | |
|----|-------------|
| 1. | 프로젝트 개요 |
| 2. | 시스템 아키텍처 |
| 3. | 기술 스택 |
| 4. | 데이터베이스 설계 |
| 5. | 주요 기능 상세 |
| 6. | 배포 현황 |
| 7. | 개발 진행도 |
| 8. | 향후 계획 (로드맵) |
| 9. | 사업계획 요약 |

1. 프로젝트 개요

낭만포차(Nangman Pocha)는 **QR코드 기반 모바일 주문 시스템**으로, 포차·바·소규모 식음료 매장의 디지털 전환을 목적으로 개발된 풀스택 솔루션입니다. 손님은 테이블 QR코드 스캔만으로 메뉴를 보고 주문할 수 있으며, 홀직원은 POS 화면에서 실시간 주문을 수신·처리하고, 관리자는 메뉴·테이블·매출 데이터를 웹에서 직접 관리합니다.

> 핵심 가치 제안

- 주문 오류 감소: 구두 주문 -> 디지털 주문으로 전환, 수기 실수 제로화
- 인건비 절감: 주문 접수 자동화로 서빙 인원 효율화
- 데이터 기반 운영: 일·월별 매출 분석, 인기 메뉴 파악
- 빠른 도입: SaaS 방식, QR 출력만으로 즉시 운영 가능
- 확장성: 멀티 스토어 지원 아키텍처, 프린터 연동 예정

> 대상 고객

- 포차·이자카야·바·호프집 등 주류 중심 소규모 매장
- 테이블 회전율이 높고 추가 주문 빈도가 많은 매장
- POS 기기 도입 비용이 부담되는 1~5인 운영 소상공인

2. 시스템 아키텍처

낭만포차는 **Turborepo 기반 모노레포** 구조로, 4개의 독립적인 앱과 2개의 공유 패키지로 구성됩니다.
프론트엔드는 Vercel에, 백엔드와 DB는 Railway에 각각 독립 배포됩니다.

구분	앱/패키지	역할	상태
앱	customer-web	손님 모바일 주문 (Next.js 14 PWA)	[완료]
앱	pos-web	홀직원 POS (Vite + React)	[완료]
앱	admin-web	관리자 대시보드 (Vite + React)	[완료]
앱	api	백엔드 REST API (Node + Prisma)	[완료]
앱	print-agent	영수증 프린터 데몬 (Node.js)	[예정]
패키지	shared-types	공통 도메인 타입 (TypeScript)	[완료]
패키지	tsconfig	공유 TypeScript 설정	[완료]

> 데이터 흐름

- 손님: QR 스캔 -> customer-web -> REST API -> PostgreSQL
- POS: Socket.IO 구독 -> 실시간 주문 수신 -> 처리·결제
- 프린터 (예정): print-agent가 PrintJob 큐 폴링 -> ESC/POS 출력
- 관리자: admin-web -> REST API -> 메뉴·테이블·매출 CRUD

3. 기술 스택

> 프론트엔드

customer-web	Next.js 14, TailwindCSS, Zustand, PWA
pos-web	Vite 5, React 18, TailwindCSS, Zustand, Socket.IO
admin-web	Vite 5, React 18, TailwindCSS, Recharts
공통	TailwindCSS utility-first, 반응형 모바일 우선

> 백엔드

런타임	Node.js 20 (NestJS 마이그레이션 예정)
ORM / DB	Prisma 5 + PostgreSQL 16
실시간	Socket.IO (예정)
캐시	Redis (선택적)
인증	JWT (도입 예정)
결제	PortOne(아임포트) 또는 토스페이먼츠

> 인프라 / DevOps

모노레포	pnpm 9 + Turborepo 2
언어	TypeScript 5 (strict)
프론트 배포	Vercel — customer-web, pos-web, admin-web
백엔드 배포	Railway — api, PostgreSQL, Redis
CI	GitHub Actions (lint + type-check on PR)
로컬 환경	Docker Compose — PostgreSQL + Redis

4. 데이터베이스 설계

PostgreSQL 16 기반, Prisma 5 ORM으로 관리합니다. 모든 모델은 cuid() PK를 사용하며 snake_case 컬럼명을 camelCase로 매핑합니다.

모델	역할	주요 필드
Store	매장 마스터	name, businessNumber, phone, address
Staff	직원 계정	email, passwordHash, role(OWNER/MANAGER/SERVER)
RestaurantTable	테이블	tableNumber, qrToken(unique), capacity, status
MenuCategory	메뉴 카테고리	name, displayOrder, isActive
Menu	메뉴 항목	name, price, description, imageUrl, isSoldOut
MenuOption	옵션 그룹	name, isRequired, maxSelect
MenuOptionItem	옵션 선택지	name, additionalPrice
Order	주문	status, type(CUSTOMER_MOBILE/POS), totalAmount
OrderItem	주문 항목	quantity, unitPrice, status, selectedOptions
Payment	결제	method, status, amount, approvalNumber
PrintJob	프린트 큐	type(KITCHEN/HALL), status, payload
DailySales	일별 매출	date, totalRevenue, orderCount, closedBy

> 열거형 (Enum)

- StaffRole: OWNER | MANAGER | SERVER
- TableStatus: AVAILABLE | OCCUPIED | RESERVED
- OrderStatus: PENDING | CONFIRMED | PREPARING | SERVED | COMPLETED | CANCELLED
- PaymentMethod: CARD | CASH | KAKAO | NAVER | TOSS
- PaymentStatus: PENDING | PAID | FAILED | REFUNDED | PARTIAL_REFUNDED
- PrintJobStatus: QUEUED | PRINTED | FAILED

5. 주요 기능 상세

> 손님 모바일 (customer-web)

- QR 토큰 진입 — /t/[tableToken] 라우트로 테이블 자동 인식
- 메뉴 카테고리 탭 필터링 + 품질 메뉴 비활성화 표시
- 메뉴 상세 바텀시트 — 옵션 선택, 수량 조절, 요청사항 입력
- 장바구니 플로팅 버튼 + 장바구니 화면
- 주문 제출 -> 주문 완료 확인 화면 (/order/[orderId])
- PWA 지원 — 홈 화면 추가 가능, 오프라인 스플래시

> 홀직원 POS (pos-web)

- 테이블 그리드 뷰 — 상태별 색상(빈 자리 / 착석 / 예약) 표시
- 실시간 주문 수신 — Socket.IO 연결, 신규 주문 토스트 알림
- 주문 패널 — 테이블별 주문 내역, 항목 확인 / 취소
- 직접 주문 입력 — 메뉴 피커 다이얼로그로 POS 주문 추가
- 결제 다이얼로그 — 결제 수단 선택, 할인 적용, 테이블 초기화
- 합산 금액 실시간 갱신

> 관리자 (admin-web)

- 대시보드 — 오늘 매출, 주문 수, 테이블 현황 통계 카드
- 메뉴 관리 — 카테고리 CRUD, 메뉴 CRUD, 품질 토글
- 테이블·QR 관리 — 테이블 추가/수정, QR코드 생성·인쇄
- 일별 매출 — 날짜별 목록, 결제 수단 분포
- 월별 매출 — Recharts 라인 차트, 월간 합산
- 모바일 하단 네비게이션 + PC 사이드바 반응형 레이아웃

> 백엔드 API (api)

- RESTful API — Store, Menu, Table, Order, Payment, DailySales
- Prisma 5 + Railway PostgreSQL — 마이그레이션 자동 배포
- 시드 데이터 — 샘플 매장·카테고리·메뉴·테이블·QR토큰
- 헬스체크 — GET /health -> {status:'ok', db:'ok'}
- 환경 분리 — NEXT_PUBLIC_USE MOCK / VITE_USE MOCK 플래그

6. 배포 현황

프론트 3개 앱은 Vercel에, 백엔드 API + PostgreSQL은 Railway에 배포 · 운영 중입니다.

앱	플랫폼	URL / 도메인	상태
customer-web	Vercel	nangman-customer-web.vercel.app	[운영중]
pos-web	Vercel	nangman-pos-web-o3p6.vercel.app	[운영중]
admin-web	Vercel	nangman-admin-web.vercel.app	[운영중]
api	Railway	nangman-production.up.railway.app	[운영중]
PostgreSQL	Railway	Railway 내부 네트워크	[운영중]
print-agent	매장 PC	클라우드 미배포 — 로컬 데몬	[미구현]

> CI/CD 파이프라인

- GitHub PR -> GitHub Actions lint + type-check 자동 실행
- main 브랜치 푸시 -> Vercel 자동 배포 (turbo-ignore로 변경 앱만 재배포)
- main 브랜치 푸시 -> Railway 자동 빌드 + prisma migrate deploy

> 검증된 API 엔드포인트

- GET /health -> {"status":"ok","db":"ok"}
- GET /stores/store-001/menu -> 카테고리 + 메뉴 JSON 반환 정상 확인

7. 개발 진행도

[완료]된 작업

단계	항목	완료 내용
1단계	시스템 아키텍처 설계	모노레포 구조, 앱 분리, 기술 스택 확정
2단계	DB ERD 설계	13개 모델, Enum 6종, Prisma 스키마 완성
3단계	손님 모바일 (customer-web)	QR 진입, 메뉴 탭, 장바구니, 주문 제출 전체 플로우
4단계	홀직원 POS (pos-web)	테이블 그리드, 실시간 주문 수신, 결제 처리
모노레포	통합 + 배포 설정	Vercel · Railway 셋업, turbo 빌드, CI 구성
6단계	관리자 (admin-web)	대시보드, 메뉴 · 테이블 관리, 매출 조회, QR 인쇄
백엔드	DB 셋업 + 운영 배포	Prisma + Railway PostgreSQL, 마이그레이션 · 시드

[보류] 다음 작업

단계	항목	내용
5단계	영수증 프린터 연동 *	ESC/POS print-agent 데몬, PrintJob 큐 — 최우선
6단계+	관리자 기능 확장	옵션 그룹 UI, 마감 처리, JWT 권한 분리, 직원 · 매장 설정
전체	Socket.IO 전면 도입	실시간 주문 상태 동기화 고도화
API	NestJS 마이그레이션	순수 Node http -> NestJS 모듈 구조
결제	PG사 실결제 연동	PortOne(아임포트) 또는 토스페이먼츠

> 전체 완성도 — 약 58% (7 / 12 단계 완료)



58% 완료

8. 향후 계획 (로드맵)

> Phase 1 — 프린터 연동 (5단계, 최우선)

- apps/print-agent 데몬 구현 (Socket.IO + 큐 폴링)
- ESC/POS 템플릿 2종: kitchen.ts (주방표) / receipt.ts (영수증)
- PrintJob 모듈·DB 추가 -> order.created / payment.approved 이벤트 enqueue
- 프린터 시뮬레이터(콘솔 출력) 모드 선검증 후 실기기 테스트
- printers.json 기반 교체 가능한 프린터 설정 구조

> Phase 2 — 관리자 기능 확장

- 메뉴 옵션 그룹·항목 CRUD UI (/menus/options)
- 마감 처리 화면 — DailySales 스냅샷 생성 (/sales/closing)
- JWT 도입 + OWNER/MANAGER/SERVER 권한 가드
- 직원 계정 관리 (/staff), 매장 설정 (/settings)
- 카테고리 드래그 정렬 (현재 ↑ ↓ 버튼)

> Phase 3 — 실시간 고도화 + 결제 연동

- Socket.IO 전면 도입 — 주문 상태 실시간 동기화
- PG사 실결제 연동 (PortOne 또는 토스페이먼츠)
- NestJS 마이그레이션으로 구조화된 백엔드
- Redis 캐시 본격 활용

> Phase 4 — SaaS 확장

- 멀티 스토어 지원 — 스토어 생성·구독 플로우
- 요금제 도입 (Freemium / Pro / Business)
- Analytics 대시보드 고도화
- PWA -> 네이티브 래퍼 (Capacitor 등) 검토

9. 사업계획 요약

> 시장 기회

국내 소규모 식음료 매장은 약 70만 개 이상이며, 그 중 포차·주점류는 약 15만 개로 추산됩니다. 기존 POS 솔루션의 초기 도입비(50~200만원) + 월정액(3~10만원)은 소상공인에게 높은 장벽이며, QR 주문 특화 제품도 포차·주류 매장에 최적화된 사례가 드뭅니다.

> 수익 모델

요금제	월 정액	포함 기능
Starter	무료	1개 매장, 테이블 5개, 기본 통계
Pro	29,000원	테이블 무제한, 프린터 연동, 고급 통계
Business	79,000원	멀티 스토어 3개, 전 기능, 전용 지원

> 목표 KPI (12개월)

지표	3개월	6개월	12개월
가입 매장 수	50개	200개	1,000개
유료 전환율	10%	15%	20%
MRR (월 반복매출)	15만원	90만원	580만원
일 평균 주문 수	500건	3,000건	20,000건

> 경쟁 우위

- 포차·주류 특화: 술집 메뉴 구조(소주·맥주·안주) + 추가 주문 특화 UX
- 저비용 도입: 하드웨어 불필요, QR 출력만으로 당일 운영 시작
- 빠른 개발 사이클: 모노레포 + CI/CD로 기능 추가·배포 속도 우위
- 오픈소스 스택: 벤더 종속 없이 유연한 확장

> 운영 비용 추산 (월)

항목	비용	비고
Vercel Pro	\$20/월	커머셜 사용 시 (Hobby 무료)

Railway	\$5~20/월	API + PostgreSQL + Redis
도메인	약 1,000원/월	연간 약 12,000원
합계	약 4~5만원/월	초기 운영 기준